03 UI 程式設計

黃彬華編撰

- UI 設計模式
- 何謂 Composable？
- Composable 運作原理
- 畫面內容
- 取用非程式資源
- 預覽畫面
- 文字方塊
- 文字輸入方塊
- 狀態屬性
- 容器元件
- 按鈕
- 分隔元件
- 元件之間的覆蓋
- 元件的顯示與隱藏
- 元件的座標與尺寸資訊
- 深色模式
- Snackbar 簡短通知
- Icon 與 Image 導論
- Icon 載入圖示
- Image 載入圖片

UI 設計模式

黃彬華編撰

- Compose UI
  - Jetpack Compose 屬於 Kotlin APIs
  - 屬於 declarative UI (宣告式 UI)
  - 僅支援 Kotlin 語言，不支援 Java 語言
  - 本課程採用 Compose UI
- Views-based UI
  - 早期採用，目前仍支援

何謂 Composable？

黃彬華編撰

- Composable 函式內容可建構局部的 UI 畫面；換句話說，UI 畫面可以由多個 Composable 函式組成
- 函式加上 @Composable 即為 Composable 函式
- Composable 函式只能被另一個 Composable 函式呼叫

Composable 運作原理

黃彬華編撰

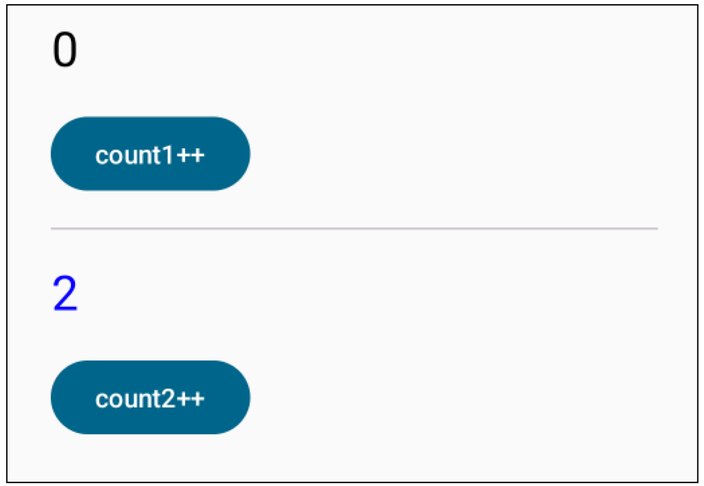
- 初始組合 (Initial Composition)：
  - 當 composable 函式第一次被呼叫時，會執行並描述對應 UI 內容
  - 這會根據可組合項創建一棵 UI 元素樹
- 重新組合 (Recomposition)：
  - 當 composable 函式的狀態 (例如狀態變數) 發生變化時，compose 會重新組合 (recomposition)；但只有依賴於已更改狀態 (例如狀態變數值改變) 的 UI 部分會更新，從而有效地更新 UI
    - 例如狀態變數 (State<T> 物件) 值改變時，composable 函式會重新被執行，但只有顯示 (依賴) 該變數的 Text 元件會更新

畫面內容

- AndroidManifest.xml > application > activity
  - android:name=".MainActivity"
    - 代表將 MainActivity 設為首頁
- MainActivity.kt 內容
  - 呼叫 setContent() 以設定 activity 的 root view
    - 呼叫 Theme() 以建立預設的佈景主題 (theme)

範例 ComposableDemo

黃彬華編撰



- 點擊各別按鈕以觀察 composable 運作原理

取用非程式資源

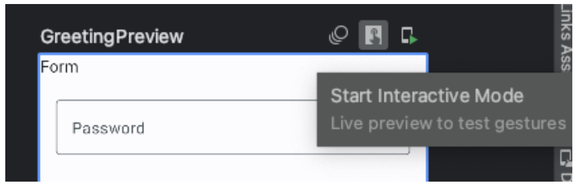
黃彬華編撰

- Android 專案的 res 目錄內存放著許多非程式資源，方便 app 取用，無須再用複雜的 I/O 方式讀入
  - 例如：文字、顏色、圖形
- 取用資源方式如下
  - 文字：stringResource(R.string.string_name)
  - 顏色：colorResource(R.color.color_name)
  - 圖形：painterResource(R.drawable.image_name)

預覽畫面

黃彬華編撰

- @Preview 用來建立預覽畫面
  - 如果需要在預覽畫面中顯示狀態和動作列，使用 showSystemUi 參數
    - @Preview(showSystemUi = true)
  - 顯示 dark mode 需使用 uiMode 參數
    - @Preview(uiMode = Configuration.UI_MODE_NIGHT_YES)
- 再加上 @Composable 來指定想要預覽的畫面
- 右上角可點選 Split 或 Design 來顯示預覽畫面
  - 如果沒有顯示 Split 或 Design 按鈕，關閉該 activity 檔案後重啟就會顯示
  - 開啟互動功能：點擊預覽畫面右上方「Start Interactive Mode」按鈕
- 參看 預覽 UI



範例 ComposablePreviewDemo

黃彬華編撰



- 顯示各種設定的預覽畫面

文字方塊 - 字型設定

黃彬華編撰

```
Text(
    text = "Form",
    fontSize = 30.sp, // 字型大小: 30
    fontWeight = FontWeight.Bold, // 字體樣式: 粗體
    color = Color.Blue // 字的顏色: 藍色
)
```

文字方塊 - 樣式設定

黃彬華編撰

- 圓角設定
  - shape = RoundedCornerShape(16.dp)
- 文字對齊方式
  - 文字對齊末端："textStyle = TextStyle(textAlign = TextAlign.End)"
- 元件內部距離
  - padding(16.dp)：上下左右內部距離皆為16
  - padding(top = 16.dp)：上內部距離為16
  - padding(horizontal = 16.dp, vertical = 32.dp)：水平內部距離為 16；垂直為 32
  - padding(start = 16.dp, top = 64.dp, end = 8.dp, bottom = 32.dp)：上下左右設定不同內部距離

文字輸入方塊

黃彬華編撰

- TextField
  - 無外框文字輸入方塊
- OutlinedTextField
  - 有外框文字輸入方塊
- 呼叫 text field 建構式
  - 可設定為密碼輸入方塊
    - visualTransformation = PasswordVisualTransformation()
  - 當 text field 輸入值改變時呼叫指定方法，並將值傳給自動宣告的變數 "it"
    - onValueChange = { uid = it }

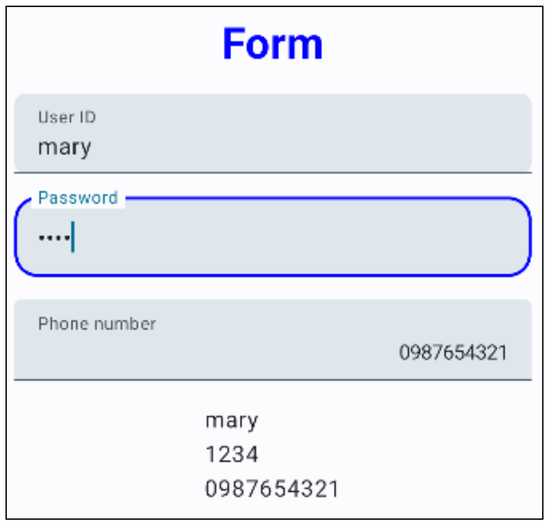
狀態屬性

黃彬華編撰

- Compose UI 使用狀態屬性 (state property) 來控制哪些 view 需要重繪
- 宣告狀態屬性
  - var password by remember { mutableStateOf("") }
- View 的屬性 (例如 text) 可以加上單向或雙向 binding
  - 單向：接收資料更新並顯示
    - Text(text = password)
  - 雙向：利用 onValueChange 參數設定，輸入值更改會同時更改狀態屬性的值
    - TextField(value = password, **onValueChange** = {password = it})

範例 TextDemo

黃彬華編撰



- 輸入帳號、密碼、電話會顯示在下方
- 點選密碼輸入時會顯示藍色框線
- 點選電話輸入時會彈出電話鍵盤
- 預覽畫面如果看不到變化，請改用模擬器

容器元件

黃彬華編撰

- 使用容器的好處
  - 群組化，方便群組移動 (所有容器都有此好處)
  - 子元件會依照容器版面配置特性自動排列好
- Column 容器
  - 子元件呈垂直排列
  - 建構式參數設定
    - horizontalAlignment：內部子元件的水平對齊方式，預設為 Alignment.Start
    - verticalArrangement：內部子元件之間的垂直間隔空間大小
- Row 容器
  - 子元件呈水平排列
  - 建構式參數設定
    - verticalAlignment：內部子元件的垂直對齊方式，預設為 Alignment.Top
    - horizontalArrangement：內部子元件之間的水平間隔空間大小

按鈕

黃彬華編撰

- 按鈕有 Button 與 TextButton
  - 預設 Button 有背景色，TextButton 則無
- 要建立按鈕，呼叫建構式設定點擊事件處理與按鈕上的圖文
  - onClick：填入點擊事件處理程式碼
  - content：設定按鈕上的圖文
    - Text：建立文字方塊
    - Icon：建立圖示，可以使用內建圖示
    - Image：使用自訂圖示

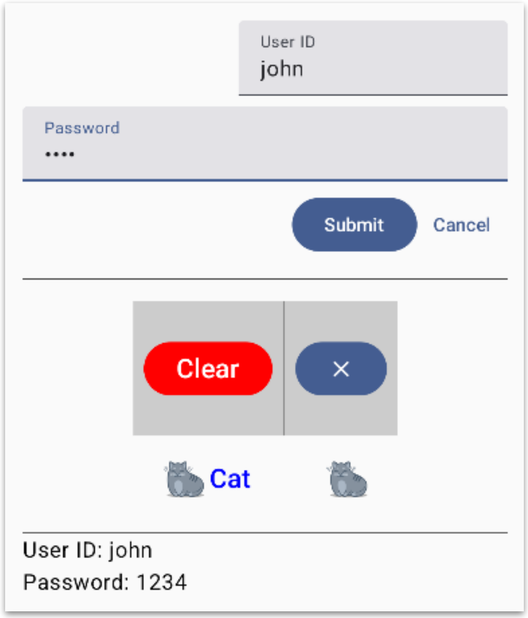
分隔元件

黃彬華編撰

- Spacer
  - 填補空白內容
  - 可用來分隔其他元件
- HorizontalDivider
  - 水平分隔線
- VerticalDivider
  - 垂直分隔線

範例 UiDemo

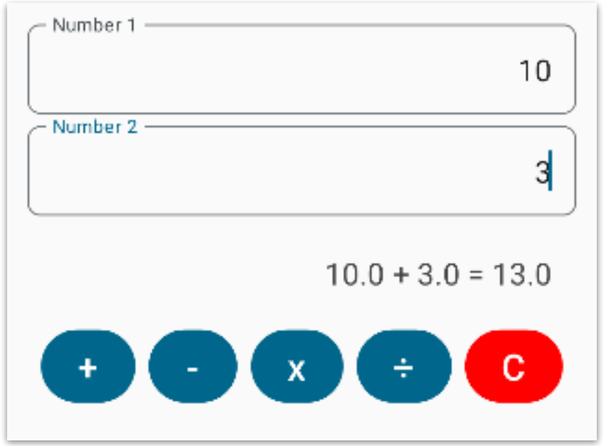
黃彬華編撰



- 輸入完資料後點擊 Submit 按鈕會在下方顯示輸入結果，點擊 Cancel 按鈕會清除結果
- 點擊 Clear 按鈕會清除結果文字
- 點擊 Cat 按鈕會顯示 "Cat"

Ex 3.1

黃彬華編撰



- 撰寫一個簡易型計算機
- 觸擊文字輸入方塊會跳出數字鍵盤
- 輸入數字後按 "+", "-", "x", "÷" 會計算對應結果並顯示
- 按 "C" 會清除所有文字

元件之間的覆蓋

黃彬華編撰

- Box 容器
  - 子元件由下至上層疊在一起，先擺放的元件會被後擺放的元件覆蓋
  - contentAlignment：內容物對齊方式
  - 子元件可以透過 "modifier = Modifier.align(Alignment.TopCenter)" 來指定對齊父容器方式；如果沒指定，就會依照上述 contentAlignment 方式來對齊

範例 BoxDemo

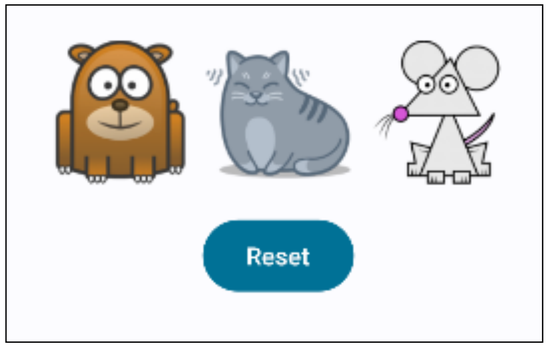


- 依照不同的設定方式，會造成圖片與文字之間的覆蓋方式不同

元件的顯示與隱藏

- 可以使用狀態屬性設定是否要顯示
  - 如果狀態屬性為要顯示，就建立 View 元件
  - 如果狀態屬性為要不要顯示，就不要建立 View元件

範例 ViewHideDemo



- 點擊畫面上的任何圖示都會將該圖示隱藏
- 點擊 Reset 按鈕會將被隱藏的所有圖示重新顯示出來

元件的座標與尺寸資訊

黃彬華編撰

- 呼叫 Modifier.onGloballyPositioned() 可以取得該元件的原點座標與尺寸資訊
  - 子元件即可以依照該資訊做進一步相對性排版，例如子原件寬度為父元件的 20%
- 取得目前 composable 元件的 density (密度，是 Android UI 元件在不同螢幕尺寸的縮放依據)
  - val density = LocalDensity.current
- with(density) 區塊可以將目前元件尺寸由 px (pixels) 轉成 dp (density-independent pixels)
  - with(density) { it.size.width.toDp() }

範例 ViewSizeDemo

黃彬華編撰



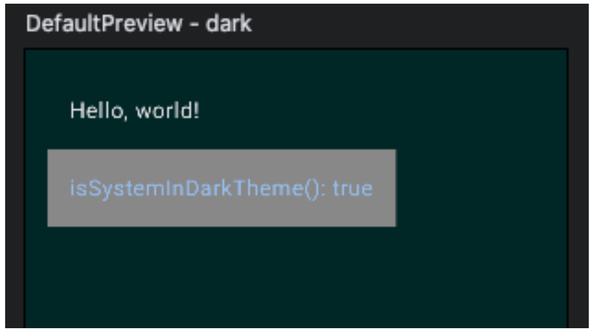
- A, B, C 文字方塊依照父元件寬、高度的 20%, 30%, 50% 比例來排版

深色模式

黃彬華編撰

- 建立雙色模式
  - Theme.kt 檔案內
    - DarkColorScheme 設定深色模式
    - LightColorScheme 設定淺色模式
    - 深色、淺色都可採用原先 Color.kt 檔案內的顏色名稱
  - 可以搭配 Surface 容器使用
    - 容器元件 Surface 的 color 為 colorScheme.surface
    - 非容器子元件未設定顏色，會自動套用 colorScheme.onSurface
  - 呼叫 isSystemInDarkTheme() 可以檢測目前模式
- 預覽畫面可以設定為深色模式
  - @Preview(name = "dark", **uiMode** = Configuration.UI_MODE_NIGHT_YES)

範例 DarkModeDemo



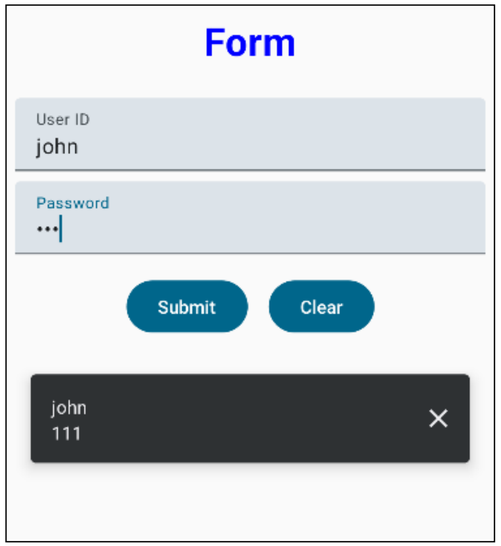
- 螢幕切換深色 / 淺色模式時，文字與背景顏色也會跟著變換

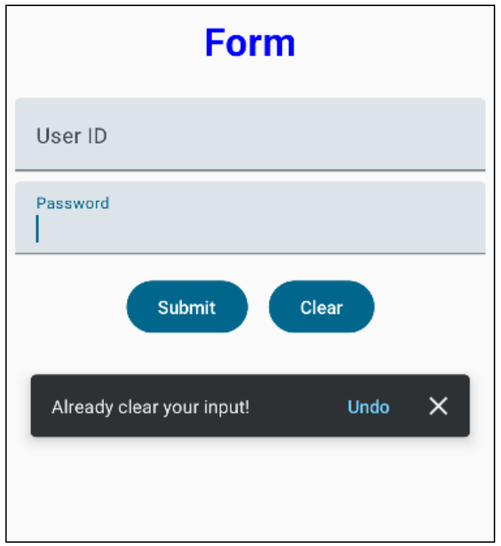
Snackbar 簡短通知

黃彬華編撰

- Snackbar 元件可做為顯示在畫面底部的簡短通知
  - 最常用於動作確認 (例如：資料已經存檔) 或狀態通知 (例如：已經離線)
  - 不會中斷使用者體驗
  - Snackbar 會在幾秒後消失，也可設定透過按鈕關閉
- 使用 Snackbar
  - 建立 SnackbarHost 以設定 Snackbar 顯示位置
  - 設定 SnackbarHostState 屬性
  - 呼叫 showSnackbar() 會改變 SnackbarHostState 狀態並顯示 Snackbar
    - 呼叫此暫停函式需要使用 CoroutineScope (可使用 rememberCoroutineScope 建立)

範例 SnackbarDemo





- 輸入帳號、密碼後
  - 點擊 Submit 按鈕會在下方彈出 Snackbar 並顯示輸入結果
  - 點擊 Clear 按鈕會清除輸入；但再點擊 Snackbar 的 Undo 按鈕會還原輸入

Icon 與 Image 導論

黃彬華編撰

- Icon 與 Image 都是用來載入圖形檔案並呈現給使用者觀看
  - Icon 適用於小型圖示
  - Image 適用於大型圖片

Icon 載入圖示

黃彬華編撰

- 圖示來源
  - Vector Asset
    - res 目錄右鍵選 New > Vector Asset > 點擊 Clip art 圖示 > Select Icon 視窗 > 選擇想要圖示後點擊 OK 按鈕 > 點擊右下角 Next 按鈕 > 指定存放目錄 (預設為 drawable) > 點擊 Finish 按鈕加入圖示
    - 呼叫 Icon() 設定 painter 屬性
      - painter = painterResource(id = R.drawable.baseline_home_24)
  - 系統圖示
    - 呼叫 Icon() 設定 imageVector 屬性
      - imageVector = Icons.Default.Email
  - 自訂圖示
    - 複製圖示檔案到 drawable 目錄
    - 呼叫 Icon() 設定 painter 屬性
      - painter = painterResource(id = R.drawable.cat)
- Icon 有 tint 屬性可設定顏色，預設為黑色

範例 IconDemo

黃彬華編撰



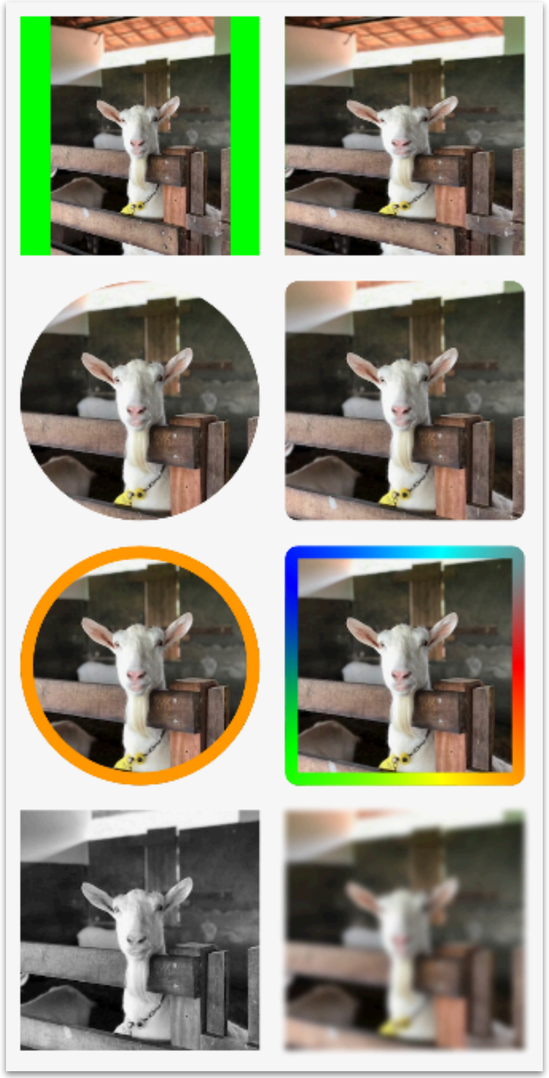
- 使用 Icon 顯示不同來源圖示

Image 載入圖片

黃彬華編撰

- Image 可以做以下設定
- 內容縮放
  - Image 的 contentScale 可設定圖片在圖框內的縮放方式，預設為 Fit (等比例縮放)
  - 參看 內容縮放
- 裁剪形狀
  - Modifier.clip() 可將圖片剪裁為指定形狀
  - 參看 將 Image 可組合項裁剪為形狀
- 加邊框
  - 組合 Modifier.border() 與 Modifier.clip()，在圖片周圍建立邊框
  - 參看 為 Image 可組合項加上邊框
- 色彩濾鏡
  - Image 的 colorFilter 可變更圖片個別像素的輸出內容
  - 參看 色彩濾鏡
- 模糊處理
  - Modifier.blur() 提供 radiusX 和 radiusY 來分別指定水平和垂直方向的模糊半徑
  - 參看 將 Image 可組合項模糊處理

範例 ImageDemo



- Image 顯示圖片時採用不同設定